WATT-WATCH — Person 1 guide: Database & Billing (Team lead)

> **You are Person 1, the team lead.** You run the app on your laptop, own the database and bill calculation, collect your teammates' results, and put the final report together.
>
> **Deadline: 2 days.** Follow the steps in order. Each step says **what to do**, **why**, and **what you should see**.

---

1. What is WATT-WATCH? (read this first)

In Tamil Nadu, electricity is billed in **slabs**: the more units you use in a billing cycle (about 2 months), the higher the price of each extra unit. Families usually find out how much they used only when the bill arrives.

WATT-WATCH is a **web application** (it runs in a browser) that:

```
Scan meter → Track units → Predict bill → Warn before a costlier slab → Reduce usage → Se
e savings
```

It has **5 tabs**: 🏠 Households · 📷 Scan & Track · 📈 Predict & Warn · 💡 Reduce & Save · 🧾 Bills. All data is stored in an **SQLite database** file: `data\wattwatch.db`.

The team:

Person	Role
You (Person 1)	Runs the app · database · tariff & bills · final report
Person 2	Meter photos (OCR) · prediction · early warning
Person 3	Appliances · saving plan · slides · demo video

---

2. Your job at a glance

When	Task	Time	Hand in
Day 1 morning	Step A: Install & run the app	1 h	App running
Day 1 morning	Step B: Team meeting & walkthrough	1 h	Google Doc created
Day 1 afternoon	Step C: Understand your code	45 min	—

When	Task	Time	Hand in
Day 1 afternoon	Step D: Check the tariff	1 h	Tariff source + rates
Day 1 afternoon	Step E: Bill check with real bills	30 min	Bill check table
Day 1 afternoon	Step F: Database screenshots & SQL	45 min	6 screenshots
Day 1 afternoon	Step G: ER diagram	30 min	ER diagram PNG
Day 1 afternoon	Step H: Add 3 homes + test teammates' data	1 h	Prediction screenshots, OCR results
Day 1 evening	Step I: Write report section 7.1	1–2 h	Section 7.1
Day 2	Step J: Assemble report, practise	full day	Final report

---

Step A — Install and run the app (1 hour)

> **Already have the app running from `Documents\WATT-WATCH`?** Skip to Step A6.

A1. Install Python

1. Open **python.org** → **Downloads** → click the yellow **Download Python** button.
2. Open the downloaded file.
3. ⚠️ **Tick "Add python.exe to PATH"** at the bottom of the first screen. *(Why: so Windows can find Python when you type `python`.)*
4. Click **Install Now** → wait → **Close**.

A2. Install Git

1. Open **git-scm.com** → **Download for Windows** → open the file.
2. Click **Next** on every screen, then **Install** → **Finish**.

A3. Open Command Prompt Press the **Windows key**, type **cmd**, press **Enter**. A black window opens. *(Use Command Prompt, not PowerShell — PowerShell can block the next steps.)*

A4. Check the installs — type each line, press **Enter**:

```
python --version
git --version
```

✅ You should see something like `Python 3.12.x` and `git version 2.x`. ❌ "not recognized" → reinstall that program (for Python, tick the PATH box). If only `python` fails, try `py --version` and use `py` instead of `python` everywhere below.

A5. Download the code — type these lines one at a time:

```
cd %USERPROFILE%\Documents
git clone -b claude/code-implementation-guide-fqx9ks https://github.com/ZumairaSultana/WA
TTWATCH.git WATT-WATCH
cd WATT-WATCH
```

- `cd` = "go into a folder".
- `git clone` = download the project from GitHub into a folder called **WATT-WATCH**.
- If a window asks you to sign in to GitHub, sign in.

A6. Create the virtual environment and install packages

```
cd %USERPROFILE%\Documents\WATT-WATCH
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

- `venv` = a private box for this project's Python packages.
- After `activate` you see `(venv)` at the start of the line. ⚠️ You must activate it **every time** you open a new Command Prompt.
- `pip install` downloads Streamlit, pandas, etc. (takes a few minutes).

A7. Run the tests

```
python -m pytest
```

✅ The last line must say `32 passed`. *(The tests automatically check the calculations and the database.)*

A8. Start the app

```
streamlit run app.py
```

✅ Your browser opens at **http://localhost:8501** showing WATT-WATCH.

- Keep the Command Prompt window open while using the app.
- To stop the app: click the Command Prompt window, press **Ctrl + C**.

Every time after today, starting the app is just:

```
cd %USERPROFILE%\Documents\WATT-WATCH
venv\Scripts\activate
streamlit run app.py
```

A9. Install Tesseract (needed to read Person 2's meter photos)

1. Search the web for **"Tesseract UB Mannheim"** → download the Windows 64-bit installer → install with default options.

2. Add it to PATH so the app can find it:
    - Windows key → type **"environment variables"** → open **Edit the system environment variables**.
    - Click **Environment Variables…** → under *User variables* select **Path** → **Edit** → **New**.
    - Type the install folder, usually `C:\Program Files\Tesseract-OCR` → **OK** → **OK** → **OK**.
3. **Close and reopen** Command Prompt, then check: `tesseract --version`. ❌ Still not working? The app works without it — you just type readings instead of scanning.

Step B — Team meeting (1 hour, all 3 together)

B1. Create the shared report

1. Open **docs.google.com** → **Blank document** → name it **WATT-WATCH Report**.
2. **Share** → add both teammates' emails → set to **Editor** → **Send**.
3. Paste these headings:

```
1. Abstract
2. Introduction (the problem)
3. Objectives
4. Existing system
5. Proposed system (WATT-WATCH and its flow)
6. System design (architecture, ER diagram, database tables)
7. Modules
   7.1 Database & billing        (Person 1)
   7.2 Scanning & prediction     (Person 2)
   7.3 Saving plan & interface   (Person 3)
8. Results
9. Limitations
10. Future work
11. Conclusion
12. References
```

B2. Walk your teammates through the app (share your screen or sit together):

1. 🏠 **Households** — "These are 10 sample households from the database PDF."
2. Sidebar → **Select household** → **H001 - Arun**.
3. 🧾 **Bills** → **Generate bill for this cycle** → a bill for **150 units = Rs 240** appears. *(150 units: first 100 free, next 50 × Rs 4.80 = Rs 240.)*
4. 📷 **Scan & Track** → type **1480** in *Meter reading* → **Add reading**.
5. 📈 **Predict & Warn** → predicted units, bill and warning.
6. 💡 **Reduce & Save** → tick *Air conditioner*, *Water heater*, *Television* → a saving plan appears.

> Want to start again with fresh data? Sidebar → **Database** → tick **I understand** → **Reset to sample data**.

B3. Agree on what each person sends you today:

- Person 2 → 10 meter photos + the real reading of each + their home's readings.

- Person 3 → appliance wattage table + their home's readings and appliance hours.

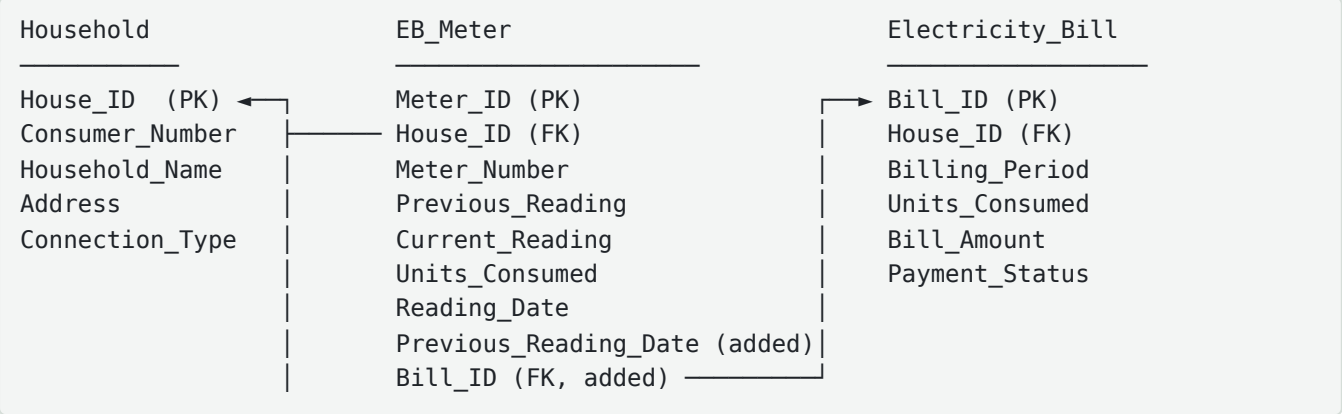
Step C — Understand your code (45 minutes)

Open the project folder in **VS Code** (**File → Open Folder → Documents\WATT-WATCH**). Your files are inside the `wattwatch` folder:

File	What it does
`schema.sql`	Creates the 3 database tables
`sample_data.py`	The 10 sample households from the PDF
`db.py`	Saves and reads households, readings and bills
`config.py`	The tariff slab rates
`tariff.py`	Calculates a bill from units

C1. The database (3 tables)

A **database** stores data in **tables** (like Excel sheets) that are linked. We use **SQLite** — the whole database is one file, built into Python, no server needed.



- **PK = Primary Key**: the unique ID of each row (no two households share a `House_ID`).
- **FK = Foreign Key**: a column that points to a row in another table (`EB_Meter.House_ID` says whose reading it is).
- **One household → many readings → many bills.**

Why we added 2 columns (you must be able to explain this):

Column	Why
`Previous_Reading_Date`	The PDF only has the date of the current reading. To calculate *units per day* we also need the start date.
`Bill_ID`	Shows which bill a reading belongs to. **Empty (NULL) = not billed yet = the current cycle.**

C2. How a reading becomes a bill (`db.py`)

1. **Add reading** → `add_reading()` copies the *previous* reading from the last row and saves a new row. `Units_Consumed = Current_Reading − Previous_Reading` is **calculated by the code, never typed** (this prevents mistakes). It also refuses a reading that is **lower** than before (a meter only counts up) or **not after** the previous date.
2. **Predict** → `current_cycle()` takes all rows with **no Bill_ID** — that's the current cycle.
3. **Generate bill** → `generate_bill()` adds up those rows' units, calculates the amount, saves one `Electricity_Bill` row, and writes its `Bill_ID` onto those readings. The next cycle starts from the last reading.

C3. The sample data (`sample_data.py`)

The 10 households (Arun … Vijay) come from the PDF. **The PDF has no dates**, so we **assumed** each sample reading covers **1 Aug – 30 Sep 2026**. The data is **fictional**. Say both of these in the report.

C4. The bill calculation (`tariff.py`, `config.py`)

Slabs are **telescopic**: each unit is charged at the rate of the slab it falls in.

`config.py` holds the slab table (an **example** — you will verify it in Step D):

```
DEFAULT_SLABS = [
    (100, 0.00),      # units 1–100: free
    (400, 4.80),      # units 101–400: Rs 4.80 each
    (500, 6.45),
    (600, 8.55),
    (800, 9.65),
    (1000, 10.70),
    (math.inf, 11.80),  # above 1000
]
```

`tariff.py` splits the units across the slabs:

```
lower = 0.0
for slab in tariff.slabs:
    if units <= lower:
        break
    in_slab = min(units, slab.upto) - lower   # units that fall in this slab
    charges.append(SlabCharge(lower, slab.upto, in_slab, slab.rate, in_slab * slab.rate))
    lower = slab.upto
```

Worked example — 450 units:

Slab	Units in slab	Rate	Amount
0 – 100	100	0.00	0.00
101 – 400	300	4.80	1,440.00
401 – 450	50	6.45	322.50
Total	**450**		**Rs 1,762.50**

Step D — Check the tariff (1 hour) ⚠️ most important

The rates above were **not confirmed** against an official source. Your job is to check them.

1. Search for the **current Tamil Nadu domestic (LT) electricity tariff** on the official website of **TNPDCL** (Tamil Nadu Power Distribution Corporation) or **TNERC** (Tamil Nadu Electricity Regulatory Commission). Look for the latest **tariff order** or **tariff schedule** for domestic consumers.
2. Write down: each slab (units range), its rate, any fixed charge, and any subsidy rule (e.g. free units).
3. Open `wattwatch\config.py` in VS Code and compare with `DEFAULT_SLABS`.
4. **If a rate is different:** change the number, press **Ctrl + S** to save, then in Command Prompt run `python -m pytest` (✅ should still say `32 passed` — the tests use their own rates).
5. Restart the app (Ctrl + C, then `streamlit run app.py`) so it uses the new rates.
6. In the report write: **the website, the date you checked, and the rates** you used.

> If the official tariff has rules our calculator doesn't handle (for example, a different price for *all* units once you cross a limit), don't change the code now — write it under **Limitations**.

Step E — Bill check with real bills (30 minutes)

1. Get **2 old electricity bills** from home (paper bill or the online account). Note the **units consumed** and the **amount** of each.
2. In Command Prompt (with `(venv)` active, in the project folder) run — replace `450` with the bill's units:

```
python -c "from wattwatch.tariff import Tariff, calculate_bill; from wattwatch import con
fig; print(calculate_bill(450, Tariff.from_rows(config.DEFAULT_SLABS)))"
```

✅ It prints the amount the app would charge, e.g. `1762.5`. 3. Fill in this table for the report:

Bill	Units	Real amount (Rs)	App amount (Rs)	Difference	Possible reason
Bill 1					
Bill 2					

Possible reasons for a difference: fixed charges, taxes or other items on the bill, an older tariff in force at that time, or subsidy rules. A difference is fine if you explain it.

Step F — Database screenshots and SQL (45 minutes)

1. Download **DB Browser for SQLite** from **sqlitebrowser.org** → install.

2. **Stop the app** (Ctrl + C in Command Prompt). *(Two programs writing the database at once can cause "database is locked".)*
3. Open DB Browser → **Open Database** → `Documents\WATT-WATCH\data\wattwatch.db`.
4. **Browse Data** tab → in *Table*, pick **Household** → screenshot. Repeat for **EB_Meter** and **Electricity_Bill**.
5. **Execute SQL** tab → paste one query → press the ▶ button (or **Ctrl + Enter**) → screenshot the result. Repeat for each:

```
-- 1. All households
SELECT * FROM Household;
```

```
-- 2. Total units recorded per household (highest first)
SELECT h.House_ID, h.Household_Name, SUM(m.Units_Consumed) AS Total_Units
FROM EB_Meter m JOIN Household h ON h.House_ID = m.House_ID
GROUP BY h.House_ID
ORDER BY Total_Units DESC;
```

```
-- 3. Bills not paid yet
SELECT * FROM Electricity_Bill WHERE Payment_Status <> 'Paid';
```

What these mean (write one line under each screenshot in the report):

- `SELECT * FROM Household` → show every column of every household.
- `JOIN` → combine readings with household names using the matching `House_ID`.
- `SUM(...) GROUP BY` → add up units for each household separately.
- `WHERE Payment_Status <> 'Paid'` → only bills that are not paid.

6. ⚠️ **File → Close Database** before starting the app again.

Step G — ER diagram (30 minutes)

An **ER (Entity-Relationship) diagram** is a picture of the tables and how they link.

1. Open **app.diagrams.net** (draw.io, free) → **Create New Diagram** → **Blank**.
2. Left panel → search **"table"** → drag in **3 tables**: Household, EB_Meter, Electricity_Bill.
3. Type the columns from Step C1 into each. Mark **PK** and **FK**.
4. Draw lines:
    - Household → EB_Meter (one household has many readings)
    - Household → Electricity_Bill (one household has many bills)
    - Electricity_Bill → EB_Meter (one bill covers many readings)
5. **File → Export as → PNG** → put it in report section 6.

Step H — Add your 3 homes and test teammates' data (1 hour)

H1. Add each home (yours, Person 2's, Person 3's):

1. You need for each home: **meter number**, the **reading and date on the last bill**, and **today's reading**.

2. Sidebar → ➕ **Add a household** → fill in House ID (keep the suggested one), consumer number, name, connection type → **Add household**. ⚠️ Use a **made-up consumer number and no address** unless the family agrees to share the real one.
3. 📷 **Scan & Track** → since it's a new household, it asks for *Meter number*, *Last bill reading date*, *Last bill reading* → fill them, then today's date and reading → **Add reading**.
4. 📈 **Predict & Warn** → screenshot. Send each teammate the screenshot of their home.

H2. Test Person 2's meter photos (needs Tesseract, Step A9):

1. 📷 **Scan & Track** → **Upload** a photo → **Read number from photo**.
2. Note what the app read (**don't click Add reading** — these are only tests).
3. Send Person 2 the list: photo name → what the app read.

H3. Saving plan for Person 3's home:

1. Select Person 3's household → 💡 **Reduce & Save**.
2. Tick their appliances, change *Watts* and *Hours/day* to their numbers.
3. Screenshot the saving plan and savings → send to Person 3.

Step I — Write report section 7.1 (evening, 1–2 hours)

Write in your own words, using these sub-headings:

7.1.1 Database design — why a database (data kept safely between uses; many households); why SQLite (built into Python, single file, no server); the 3 tables and their purpose; PK and FK; the 2 added columns and why; insert the ER diagram.

7.1.2 How data flows — add reading → units calculated → current cycle → generate bill (Step C2).

7.1.3 Sample data — 10 fictional households from the database document; dates assumed (1 Aug – 30 Sep 2026).

7.1.4 Bill calculation — telescopic slabs; the 450-unit worked example; the tariff you verified (source + date).

7.1.5 Results — the bill check table (Step E); the SQL query screenshots (Step F).

Also write report sections **1–6 and 11** on Day 2 (see Step J).

Step J — Day 2

Time	You
9:00 – 11:00	Write sections **1 Abstract, 2 Introduction, 3 Objectives, 4 Existing system, 5 Proposed system, 6 System design, 11 Conclusion**. Insert all screenshots.
11:00 – 13:00	With the team: read the whole report, fix mistakes, same font/headings everywhere. Export as PDF/Word as your college requires.
14:00 – 15:00	On your laptop, record the **backup demo video** with Person 3 (**Windows + G** → Capture → Record).

Time	You
15:00 – 17:00	Practise the full presentation **3 times**.
17:00 – 18:00	Practise the questions (Section 4 below).

Short help for the sections you write:

- **Abstract** (5–6 lines): the problem, what WATT-WATCH does, the technology, the main results.
- **Introduction**: slab billing; families learn their usage only when the bill arrives.
- **Objectives**: track usage from meter readings; predict the bill; warn before a costlier slab; suggest how to save; store data in a database.
- **Existing system**: bill shown only at the end of the cycle; no early warning; no saving plan.
- **Proposed system**: the 6-step flow and 5 tabs.
- **System design**: Python + Streamlit (screens) + SQLite (data) + Tesseract (OCR); the ER diagram.
- **Conclusion**: what you built, what worked, what you would improve.

---

3. Your part of the presentation (slides 1–6)

Slide	What to say (in your own words)
1. Title	"We are [names]. Our project is WATT-WATCH, a smart electricity bill warning and saving assistant."
2. Problem	"Tamil Nadu bills electricity in slabs. The more you use, the higher the rate. Families only see their usage when the bill comes — too late to change it."
3. Solution & flow	"WATT-WATCH follows six steps: scan, track, predict, warn, reduce, save."
4. Technology	"Python, Streamlit for the web interface, SQLite for the database, Tesseract for reading meter photos."
5. Database	Show the ER diagram. "Three tables: Household, EB_Meter and Electricity_Bill, linked by House_ID. Units are always calculated from two readings."
6. Billing	Show the 450-unit example and your bill check table. "We verified the tariff from [source] and compared with real bills."

4. Questions you may be asked

Question	Answer
Is it an app or a website?	A web application — it runs in any browser, built with Python and Streamlit, data in SQLite.
Why SQLite?	Built into Python, the whole database is one file, no server — enough for this project.

Question	Answer
What is a primary/foreign key?	Primary key: unique ID of a row. Foreign key: a column linking to another table's row.
Why calculate units instead of typing them?	Prevents mistakes: units = current − previous reading.
Why did you add two columns?	Previous_Reading_Date to calculate units per day; Bill_ID to know which readings are already billed.
Is the data real?	The 10 sample households are fictional with assumed dates; our own homes are real readings.
What if the tariff changes?	Edit the rates in `config.py` or in the sidebar — no other code changes.
How is the bill calculated?	Units are split across slabs; each part × its rate; added up (450-unit example).

5. If something goes wrong

Problem	Fix
`python` is not recognized	Reinstall Python with **Add to PATH** ticked, or use `py`
`(venv)` missing	`venv\Scripts\activate`
"running scripts is disabled"	You're in PowerShell — use **Command Prompt** (cmd)
`ModuleNotFoundError`	Activate venv, then `pip install -r requirements.txt`
`database is locked`	Close the database in DB Browser, restart the app
Need fresh sample data	Sidebar → **Database** → tick **I understand** → **Reset to sample data**
"Tesseract not found"	Check Step A9 PATH; or type readings by hand
App page shows a red error	Copy the error text and ask for help